Java for DSA

Cheat sheet for coming from C++: syntax, arrays, strings, HashMap, collections, and the patterns from Block 1. Built from your own mistakes in BASICS-JAVA.

1. Program skeleton
2. Types & overflow
3. Input / output (incl. fast I/O)
4. Strings, chars, StringBuilder
5. Arrays
6. ArrayList
7. HashMap & HashSet
8. TreeMap / TreeSet
9. Stack, Queue, Deque
10. PriorityQueue (heap)
11. Sorting & comparators
12. Math & modulo
13. Block 1 patterns
14. Error messages → meaning
15. Your personal slip list

1. Program skeleton

```
import java.util.*;

public class ClassName {                        // class: name only, NO ( )
    public static void main(String[] args) {    // String[] args goes HERE
        Scanner sc = new Scanner(System.in);

        // read -> solve -> print

        sc.close();
    }

    static int helper(int x) {                  // helpers: inside class, outside main
        return x * 2;
    }
}
```

Rules

- A `public class` must have the **same name as the file** (`PrimeCheck.java` → `public class PrimeCheck`).
- Never two classes with the same name in one folder.
- Helpers need `static` so `main` can call them. No method inside another method.
- Order of methods doesn't matter; no prototypes needed.

2. Types & overflow

C++	Java	Notes
`int`	`int`	32-bit, $\pm 2.1 \times 10^9$
`long long`	`long`	64-bit, $\pm 9.2 \times 10^{18}$; literals need `L` : `10000000000L`
`bool`	`boolean`	`bool` does not exist
`char`	`char`	16-bit; still a number inside (`'0'` = 48, `'a'` = 97)
`double`	`double`	
`INT_MAX` / `LLONG_MAX`	`Integer.MAX_VALUE` / `Long.MAX_VALUE`	also `MIN_VALUE`
`unsigned`	—	Java has no unsigned types

Casting

```
long r = (long) a * b;    // ✅ cast BEFORE multiply
long r = a * b;           // ❌ int overflow first
int x = (int) 3.9;        // 3 (truncates)
int d = (int) Math.pow(2, 10); // pow returns double
```

Widening is automatic, narrowing is not

```
long x = n;           // int -> long: fine
int d = someLong;     // ❌ compile error
int d = (int) someLong; // ✅ explicit
```

> **Use `long` when**
> sums of arrays, `i*i`, products, factorials, LCM, or anything that can pass ~2×10^9. Overflow is **silent** — no error, just a wrong answer.

3. Input / output

Read	Scanner	C++
int / long / double	`sc.nextInt()` / `sc.nextLong()` / `sc.nextDouble()`	`cin >> x`
one word	`sc.next()`	`cin >> s`
whole line	`sc.nextLine()`	`getline(cin, s)`
one char	`sc.next().charAt(0)`	`cin >> ch`
more input?	`sc.hasNextInt()` / `sc.hasNext()`	`while (cin >> x)`

> `nextInt()` **then** `nextLine()` **trap**
> — the first `nextLine()` returns `""` (leftover newline). Call one extra `sc.nextLine();` to discard it.

Output

```
System.out.println(x);          // with newline
System.out.print(x + " ");      // no newline
System.out.println(a + " " + b); // "6 36"
System.out.printf("%.2f%n", d); // formatted, like C printf
```

Fast I/O (for 10^5+ numbers — Scanner can TLE)

```
import java.io.*;
import java.util.*;

public class Main {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int n = Integer.parseInt(br.readLine().trim());
        int[] arr = new int[n];
        StringTokenizer st = new StringTokenizer(br.readLine());
        for (int i = 0; i < n; i++) arr[i] = Integer.parseInt(st.nextToken());

        StringBuilder out = new StringBuilder();     // build output, print ONCE
        for (int x : arr) out.append(x).append(' ');
        System.out.println(out);
    }
}
```

Printing inside a big loop with `System.out.println` is slow — collect into a `StringBuilder` and print once.

4. Strings, chars, StringBuilder

Java `String` is **immutable** — you cannot change it, and `s[i]` does not work.

Task	Java	C++
length	`s.length()` *(brackets)*	`s.size()`
char at i	`s.charAt(i)`	`s[i]`
compare text	`s.equals(t)` **never** `==`	`s == t`
lexicographic compare	`s.compareTo(t)` (<0, 0, >0)	`s < t`
substring [i, j)	`s.substring(i, j)` (j excluded)	`s.substr(i, len)`
find	`s.indexOf("ab")` (-1 if none), `s.contains("ab")`	`s.find`
split by spaces	`String[] w = s.trim().split("\\s+");`	stringstream
case	`s.toLowerCase()`, `s.toUpperCase()`	
to char array	`char[] c = s.toCharArray();`	
char array → String	`new String(c)`	
int ↔ String	`String.valueOf(n)`, `Integer.parseInt(s)`, `Long.parseLong(s)`	`to_string`, `stoi`
binary	`Integer.parseInt("1101", 2)`, `Integer.toBinaryString(13)`	

char arithmetic

```
int digit = s.charAt(i) - '0'; // '7' -> 7
int idx   = c - 'a';           // 'c' -> 2
char ch   = (char) ('a' + 2);  // 'c'

int[] cnt = new int[26];       // letter count
for (int i = 0; i < s.length(); i++)
    cnt[s.charAt(i) - 'a']++;
```

Works because `'0'..'9'` and `'a'..'z'` are consecutive codes.

Character helpers

```
Character.isDigit(c)
Character.isLetter(c)
Character.isLetterOrDigit(c)
Character.isUpperCase(c)
Character.isLowerCase(c)
Character.toLowerCase(c)
Character.toUpperCase(c)
```

StringBuilder — the changeable string (like C++ `std::string`)

```
StringBuilder sb = new StringBuilder();
sb.append('1').append(0).append("1");  // "101" — chars, ints, strings
sb.reverse();                          // in place
sb.length();  sb.charAt(i);
sb.setCharAt(i, 'x');                  // s[i] = 'x'
sb.deleteCharAt(sb.length() - 1);      // pop_back
sb.insert(0, 'z');                     // at front (O(n))
String result = sb.toString();
```

> **Never build a string with `s = s + x` in a loop**
> — each `+` copies the whole string ($O(n^2)$). Use `StringBuilder`.

5. Arrays

```
int[] arr = new int[n];       // all 0
int[] a = {4, 2, 7};
long[] big = new long[n];
boolean[] seen = new boolean[n]; // false
String[] w = new String[n];   // null

// input
for (int i = 0; i < n; i++) arr[i] =
sc.nextInt();

// loops
for (int i = 0; i < arr.length; i++) { arr[i] }
for (int x : arr) { x }
```

```
// 2D
int[][] g = new int[rows][cols];
g.length      // rows
g[0].length   // cols
for (int[] row : g) Arrays.fill(row, -1);

// functions: passed by reference
static void change(int[] a) { a[0] = 99; }
// caller's array changes too
static int[] make(int n) { return new int[n]; }
```

Arrays.* (import java.util.*)	Does
`Arrays.sort(arr)`	ascending (int[] uses dual-pivot quicksort)
`Arrays.sort(arr, l, r)`	sort index l..r-1
`Arrays.fill(arr, -1)`	fill (1D only — loop rows for 2D)
`arr.clone()` / `Arrays.copyOf(arr, len)` / `Arrays.copyOfRange(arr, l, r)`	real copies
`Arrays.equals(a, b)`	compare contents
`Arrays.toString(arr)` / `Arrays.deepToString(g)`	debug print 1D / 2D
`Arrays.binarySearch(arr, key)`	index if found, negative if not (array must be sorted)
`Arrays.stream(arr).sum()` / `.max().getAsInt()`	quick sum / max (sum returns int — overflow risk)

Array traps

- `arr[n]` → `ArrayIndexOutOfBoundsException` (crash, not garbage like C++).
- `arr.length` — no brackets. `arr.length()` is a compile error.
- `int[] b = a;` is **not a copy** — both names point to the same array. Use `a.clone()`.
- `a == b` checks "same object", not same contents. Use `Arrays.equals`.
- Size is fixed after `new`. Need to grow? → `ArrayList`.

6. ArrayList (= vector)

```
ArrayList<Integer> list = new ArrayList<>();
list.add(5);                  // push_back
list.add(0, 7);               // insert at index
(O(n))
list.get(i);                  // v[i]
list.set(i, 9);               // v[i] = 9
list.size();
list.remove(list.size() - 1); // pop_back
list.isEmpty();
list.contains(5);             // O(n)
```

```
Collections.sort(list);
Collections.reverse(list);
Collections.max(list);  Collections.min(list);

for (int x : list) { ... }

// list of lists (adjacency list)
ArrayList<ArrayList<Integer>> adj = new
ArrayList<>();
for (int i = 0; i < n; i++) adj.add(new
ArrayList<>());
adj.get(u).add(v);
```

> `list.remove(x)` **trap**
> — with an `int` argument it removes by **index**. To remove the **value** 5: `list.remove(Integer.valueOf(5))`.
> Generics need wrapper types: `ArrayList<Integer>`, never `ArrayList<int>` (same for `Long`, `Character`, `Boolean`).

7. HashMap & HashSet (= unordered_map / unordered_set)

```
HashMap<Integer, Integer> map = new HashMap<>();      // value -> count
HashMap<Character, Integer> freq = new HashMap<>();   // char -> count
HashMap<String, Integer> wc = new HashMap<>();        // word -> count
```

C++ unordered_map	Java HashMap
`mp[k] = v;`	`map.put(k, v);`
`mp[k]` (read)	`map.get(k)` → returns **null** if missing
read with default	`map.getOrDefault(k, 0)`
`mp[k]++`	`map.put(k, map.getOrDefault(k, 0) + 1);` or `map.merge(k, 1, Integer::sum);`
`mp.count(k)`	`map.containsKey(k)` (`containsValue(v)` is O(n))
`mp.erase(k)`	`map.remove(k);`
`mp.size()` / `empty()`	`map.size()` / `map.isEmpty()`
`mp[k].push_back(v)`	`map.computeIfAbsent(k, x -> new ArrayList<>()).add(v);`

Iterating

```
for (int key : map.keySet()) { int val = map.get(key); }
for (int val : map.values()) { ... }
for (Map.Entry<Integer, Integer> e : map.entrySet()) {
    int key = e.getKey(), val = e.getValue();
}
```

`HashMap` has **no order**. Need sorted keys → `TreeMap` (section 8).

Patterns

Frequency count

```
HashMap<Integer, Integer> f = new HashMap<>();
for (int x : arr)
    f.put(x, f.getOrDefault(x, 0) + 1);
```

Most frequent element

```
int best = -1, bestCnt = 0;
for (int k : f.keySet()) {
    int c = f.get(k);
    if (c > bestCnt) { bestCnt = c; best = k; }
}
```

Decrement + remove at 0 (sliding window, so `size()` = distinct count)

```
f.put(x, f.get(x) - 1);
if (f.get(x) == 0) f.remove(x);
```

Value → index (Two-Sum style: check first, then store)

```
HashMap<Integer, Integer> idx = new HashMap<>();
for (int i = 0; i < n; i++) {
    // 1. look up what you need in idx
    // 2. then store current
    idx.put(arr[i], i);
}
```

Prefix sum + map (count subarrays with sum k)

```
HashMap<Long, Integer> cnt = new HashMap<>();
cnt.put(0L, 1);            // 0L, not 0 !
long sum = 0; int ans = 0;
for (int x : arr) {
    sum += x;
    ans += cnt.getOrDefault(sum - k, 0);
    cnt.put(sum, cnt.getOrDefault(sum, 0) + 1);
}
```

HashSet

```
HashSet<Integer> seen = new HashSet<>();
seen.add(x);          // returns false if x was already there — handy for duplicates
seen.contains(x);  seen.remove(x);  seen.size();
```

HashMap traps (Java-only)

1. `int c = map.get(k);` when `k` is missing → **NullPointerException**. Use `getOrDefault`.
2. `map.get(a) == map.get(b)` compares **objects**: correct for -128..127, **silently false** for bigger values. Store into `int` variables first, or use `.equals()`. Passes samples, fails hidden tests.
3. Key type `Long`: `map.get(0)` looks up an `Integer` 0 and finds nothing. Write `0L`.
4. Small non-negative keys (a–z, 0–1000)? An `int[]` counter is simpler and faster than a map.

8. TreeMap / TreeSet (= map / set, sorted, O(log n))

```
TreeMap<Integer, Integer> tm = new TreeMap<>();
// same put/get/getOrDefault/remove as HashMap
tm.firstKey();   tm.lastKey();     // min / max key
tm.floorKey(x);    // largest  key <= x (null if none)
tm.ceilingKey(x);  // smallest key >= x
tm.lowerKey(x);    // <  x
tm.higherKey(x);   // >  x
tm.pollFirstEntry(); // remove + return min
```

```
TreeSet<Integer> ts = new TreeSet<>();
ts.add(x);  ts.remove(x);  ts.contains(x);
ts.first();  ts.last();
ts.floor(x);  ts.ceiling(x);  // like lower_bound
ts.lower(x);  ts.higher(x);
ts.pollFirst();  ts.pollLast();
```

These can return `null` — store in `Integer` and check before unboxing.

9. Stack, Queue, Deque

Use `ArrayDeque` for all three (the old `Stack` class works but is slower).

Stack

```
Deque<Integer> st = new ArrayDeque<>();
st.push(x);       // push
st.peek();        // top (null if empty)
st.pop();         // remove + return top
st.isEmpty();  st.size();
```

Queue (BFS)

```
Queue<Integer> q = new ArrayDeque<>();
q.offer(x);       // push back
q.peek();         // front
q.poll();         // remove + return front
q.isEmpty();
```

Deque (both ends)

```
Deque<Integer> dq = new ArrayDeque<>();
dq.offerFirst(x);  dq.offerLast(x);
dq.peekFirst();    dq.peekLast();
dq.pollFirst();    dq.pollLast();
```

Pairs in a queue (grid BFS)

```
Queue<int[]> q = new ArrayDeque<>();
q.offer(new int[]{r, c});
int[] cur = q.poll();
int r = cur[0], c = cur[1];

int[] dr = {-1, 1, 0, 0};
int[] dc = {0, 0, -1, 1};
```

> `ArrayDeque` **does not allow** `null`
> — and `peek()` / `poll()` return `null` when empty, so check `isEmpty()` first.

10. PriorityQueue (heap)

```
// MIN-heap by default (opposite of C++!)
PriorityQueue<Integer> pq = new PriorityQueue<>
();

// MAX-heap
PriorityQueue<Integer> mx =
    new PriorityQueue<>
(Collections.reverseOrder());

pq.offer(x);   pq.peek();   pq.poll();
pq.size();     pq.isEmpty();
```

```
// pairs: smallest a[0] first
PriorityQueue<int[]> pq = new PriorityQueue<>(
    (a, b) -> Integer.compare(a[0], b[0]));
pq.offer(new int[]{dist, node});

// by a[0], ties by a[1]
(a, b) -> a[0] != b[0]
        ? Integer.compare(a[0], b[0])
        : Integer.compare(a[1], b[1])
```

> **C++** `priority_queue` **is a MAX-heap; Java** `PriorityQueue` **is a MIN-heap.**
> Easy to get backwards.

11. Sorting & comparators

Want	Code
int[] ascending	`Arrays.sort(arr);`
int[] descending	sort ascending, then read from the end (or reverse with two pointers)
Integer[] descending	`Arrays.sort(boxed, Collections.reverseOrder());`
2D by first column (intervals)	`Arrays.sort(iv, (a, b) -> Integer.compare(a[0], b[0]));`
ArrayList	`Collections.sort(list);` / `list.sort((a, b) -> b - a);`
Strings by length	`Arrays.sort(words, (a, b) -> a.length() - b.length());`
chars of a string	`char[] c = s.toCharArray(); Arrays.sort(c); String t = new String(c);`

> **Comparator rule:**
> return negative → `a` first, positive → `b` first.
> Prefer `Integer.compare(a, b)` over `a - b`: subtraction overflows for big/negative values (e.g. `-2e9 - 2e9`).
> A lambda comparator only works on **objects** (`Integer[]`, `int[][]`, lists) — not on `int[]`.

12. Math & modulo

```
Math.max(a, b);   Math.min(a, b);
Math.abs(x);
Math.pow(a, b);   // double! cast: (int) / (long)
Math.sqrt(x);     // double
Math.floor(x); Math.ceil(x);

// ceil(a / b) for positive ints, no doubles:
int c = (a + b - 1) / b;
```

```
final int MOD = 1_000_000_007;
long ans = 1;
for (int i = 1; i <= n; i++)
    ans = ans * i % MOD;     // long * int, then %

// negative mod fix (Java % keeps sign)
int m = ((x % k) + k) % k;   // -1 % 5 = -1 -> 4
```

13. Block 1 patterns (keep these in muscle memory)

Digit loop (sum / count / reverse)

```
int x = n;              // save original if comparing later
while (n > 0) {
    int d = n % 10;     // last digit
    rev = rev * 10 + d; // or sum += d, count++
    n = n / 10;         // drop it
}
// handle n == 0 separately if it matters!
```

Base 2 instead of 10 → decimal-to-binary bits (then `sb.reverse()`).
Binary string → decimal: `res = res * 2 + (s.charAt(i) - '0')`.

√n divisor loop (prime / count / sum divisors)

```
for (long i = 1; i * i <= n; i++) {
    if (n % i == 0) {
        // pair: i and n / i
        // if i == n / i -> count once
    }
}
// prime: start at 2, return false on first hit
```

Sieve (all primes ≤ N, N ≤ ~10^7)

```
boolean[] comp = new boolean[N + 1];
for (int i = 2; (long) i * i <= N; i++)
    if (!comp[i])
        for (int j = i * i; j <= N; j += i)
            comp[j] = true;
// primes = every i >= 2 with !comp[i]
```

GCD / LCM

```
static long gcd(long a, long b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}
long lcm = a / gcd(a, b) * b;  // divide first
```

Trailing zeros in n!

```
long count = 0;
while (n > 0) { n /= 5; count += n; }
```

Fibonacci (loop)

```
long a = 0, b = 1;
for (int i = 0; i < n; i++) {
    long next = a + b;  a = b;  b = next;
}
```

14. Error messages → what they mean

Message	Cause	Fix
The public type X must be defined in its own file	class name ≠ file name	rename class to match file
Syntax error on token "class", char expected	`class X(String[] args)`	`( )` belong on `main`, not the class
Main method not found in class X	`main()` without args	`public static void main(String[] args)`
bool cannot be resolved to a type	C++ habit	`boolean`
The type of the expression must be an array type but it resolved to String	`s[i]`	`s.charAt(i)`
Syntax error on token "*"	`a ** b` (Python)	`Math.pow` or a loop
method f(int) is not applicable for the arguments (long)	passing long to int param	make the variable `int` or cast
missing return statement	non-void method, some path without `return`	return on every path
Cannot make a static reference to the non-static method	helper without `static`	add `static`
insert ";" to complete ...	missing semicolon	add `;`
`InputMismatchException`	input type ≠ what you read (or input order swapped)	check `nextInt` vs `next` and order
`ArithmeticException: / by zero`	`n % 0` (loop started at 0)	start divisor loops at 1 or 2
`ArrayIndexOutOfBounds` / `StringIndexOutOfBounds`	`i <= n` on a 0-indexed array/string	`i < n`
`NullPointerException`	`map.get(k)` on missing key into an `int`	`getOrDefault`
`StackOverflowError`	recursion without a returning base case	base case must `return`

Yellow warnings ("*Resource leak: sc is never closed*", "*value of x is not used*", "*import never used*") don't stop the program — but "not used" often means you forgot to use or print something.

15. Your personal slip list (from Block 1)

Check these before you hit Run

1. `String[] args` inside **main's** brackets — never on the class line.
2. Class name = file name.
3. `boolean`, not `bool`. `s.charAt(i)`, not `s[i]`. No `**`.
4. Divisor loops start at 1 (count) or 2 (prime check) — never 0.
5. Include the end when it matters: `i <= n` for divisors of n; `i < n` for indexes.
6. Save the original (`int x = n;`) before a digit loop destroys `n`.
7. Test `n = 0` — `while (n > 0)` skips it.
8. Read inputs in the order the problem lists them.
9. `long` for sums, products, `i*i`.
10. Output format exactly as asked: spaces, one line vs many, `true` / `false` vs text.
11. Check constraints first: n ≤ 10^9 ⇒ O(√n) or O(log n); many queries ≤ 10^6 ⇒ precompute (sieve).

Constraint → allowed complexity (Java, ~1 s)

n up to	Target	Typical approach
10–20	$O(2^n)$, $O(n!)$	recursion / backtracking
10^3	$O(n^2)$	nested loops, 2D DP
10^5 – 10^6	O(n log n) / O(n)	sort, hashing, two pointers, prefix sums, sliding window
10^9 – 10^{18}	O(√n) / O(log n)	math, binary search, formula